

6.3 利用networkx求解图的一些基本问题

6.3.1 遍历问题

在 `networkx` 中，你可以使用内置的遍历函数来进行深度优先遍历（DFS）和广度优先遍历（BFS）。这些遍历函数允许你按照特定的顺序访问图中的节点。

深度优先遍历（DFS）

对于深度优先遍历，`networkx` 提供了 `dfs_edges` 和 `dfs_preorder_nodes` 等函数。但如果你想要一个更直观的DFS遍历，你可能需要自己实现或使用 `dfs_preorder_nodes` 来获取节点的访问顺序。

下面是一个使用 `dfs_preorder_nodes` 进行深度优先遍历的示例：

```
import networkx as nx

# 创建一个图
G = nx.Graph()
# 添加边
edges = [(1, 2), (1, 3), (2, 4), (3, 4), (4, 5)]
G.add_edges_from(edges)

# 深度优先遍历（使用dfs_preorder_nodes）
# 注意：你需要指定一个起始节点
start_node = 1
visited_nodes = list(nx.dfs_preorder_nodes(G, start_node))
print("DFS遍历顺序（从节点{}开始）:".format(start_node), visited_nodes)
```

广度优先遍历（BFS）

对于广度优先遍历，`networkx` 提供了 `bfs_edges` 和 `bfs_tree` 等函数，但最常用的是 `bfs_edges`，因为它直接给出了遍历的顺序。然而，如果你想要节点的访问顺序而不是边，你可以使用 `bfs_predecessors` 或 `bfs_successors`，或者简单地通过遍历 `bfs_edges` 来构建节点列表。

下面是一个使用 `bfs_edges` 进行广度优先遍历并构建节点访问顺序的示例：

```

import networkx as nx

# 创建一个图
G = nx.Graph()
# 添加边
edges = [(1, 2), (1, 3), (2, 4), (3, 4), (4, 5)]
G.add_edges_from(edges)

# 广度优先遍历
# 注意：你同样需要指定一个起始节点
start_node = 1
visited_nodes = []
for u, v in nx.bfs_edges(G, start_node):
    if u not in visited_nodes:
        visited_nodes.append(u)
    if v not in visited_nodes:
        visited_nodes.append(v)

# 注意：上面的代码可能不是最优的，因为它在添加节点时没有考虑到边的方向（尽管在无向图中这没关系）
# 更简洁的方法是使用集合来自动去重
visited_nodes_set = set()
for u, v in nx.bfs_edges(G, start_node):
    visited_nodes_set.add(u)
    visited_nodes_set.add(v)
visited_nodes = list(visited_nodes_set)

print("BFS遍历顺序（从节点{}开始）:".format(start_node), visited_nodes)

```

然而，`bfs_edges` 生成的遍历顺序实际上是按照边的顺序来的，因此如果你只关心节点的访问顺序而不关心它们是通过哪条边被访问的，你可能想要稍微调整上面的代码，以确保节点只被添加一次到 `visited_nodes` 列表中。上面的 `visited_nodes_set` 示例已经展示了这一点。

另外，`networkx` 的 `bfs_predecessors` 和 `bfs_successors` 函数可以提供从起始节点到每个可达节点的最短路径上的前驱和后继节点信息，这也可以用于构建BFS遍历的节点顺序，但通常需要额外的处理来得到完整的节点列表。

6.3.2 最小生成树问题

在 `networkx` 中，解决最小生成树（Minimum Spanning Tree, MST）问题的一个常用算法是Kruskal算法或Prim算法。`networkx` 提供了 `minimum_spanning_tree` 函数，该函数默认使用Kruskal算法来找到图的最小生成树。

下面是一个使用 networkx 解决最小生成树问题的代码案例：

```
import networkx as nx
import matplotlib.pyplot as plt

# 创建一个加权图
G = nx.Graph()
edges = [
    ('A', 'B', 10),
    ('A', 'C', 6),
    ('A', 'D', 5),
    ('B', 'C', 5),
    ('B', 'D', 15),
    ('B', 'E', 4),
    ('C', 'D', 15),
    ('C', 'F', 12),
    ('D', 'E', 20),
    ('E', 'F', 9)
]

# 添加带权重的边
G.add_weighted_edges_from(edges)

# 找到并绘制最小生成树
T = nx.minimum_spanning_tree(G)

# 绘制原图
plt.figure(figsize=(8, 6))
nx.draw(G, with_labels=True, node_color='lightblue', alpha=0.5, edge_color='gray')
plt.title("Original Graph")
plt.show()

# 绘制最小生成树
plt.figure(figsize=(8, 6))
nx.draw(T, with_labels=True, node_color='lightgreen', alpha=0.5, edge_colors='red', width=2)
plt.title("Minimum Spanning Tree")
plt.show()

# 打印最小生成树的边
print("Edges in the Minimum Spanning Tree:")
for u, v, weight in T.edges(data='weight'):
    print(f"{u} -- {v} weighted {weight}")
```

在这个例子中，我们首先创建了一个加权图 G ，然后使用 `nx.minimum_spanning_tree` 函数找到了这个图的最小生成树 T 。之后，我们使用 `matplotlib` 和 `networkx` 的绘图功能来分别展示原图和最小生成树。最后，我们还打印了最小生成树中所有边的信息。

注意，由于 `networkx` 的 `draw` 函数默认使用的是圆形布局，这可能导致在某些情况下，图的绘制看起来不是很清晰或直观。你可以通过调整 `draw` 函数的参数（如 `pos` 参数，它允许你自定义节点的位置）来改善这一点。然而，在这个简单的示例中，我们使用了默认的圆形布局来展示结果。

另外，`networkx` 的 `minimum_spanning_tree` 函数默认使用Kruskal算法，因为它适用于稀疏图，并且不需要事先知道图的所有顶点。如果你需要更详细的控制，或者想要使用Prim算法（它更适用于密集图），你可能需要自己实现这些算法，因为 `networkx` 的标准库中并没有直接提供Prim算法的实现作为 `minimum_spanning_tree` 的选项。不过，对于大多数实际应用来说，Kruskal算法已经足够好了。

6.3.3 最短路径问题

在 `networkx` 中，解决最短路径问题通常涉及到使用几种不同的算法，具体取决于问题的具体要求和图的特性（如有向图、无向图、是否包含负权边等）。以下是一些常用的算法和它们在 `networkx` 中的实现方式：

1. Dijkstra算法

Dijkstra算法是一种用于解决单源最短路径问题的贪心算法。它适用于带有非负权重的图。在 `networkx` 中，可以使用 `dijkstra_path` 函数来计算从源节点到目标节点的最短路径，以及使用 `dijkstra_path_length` 函数来计算最短路径的长度（即路径中所有边的权重之和）。

示例代码：

```
import networkx as nx

# 创建一个有向图
G = nx.DiGraph()
# 添加带权重的边
G.add_edge('A', 'B', weight=3)
G.add_edge('A', 'C', weight=2)
G.add_edge('B', 'C', weight=1)
G.add_edge('B', 'D', weight=4)
G.add_edge('C', 'D', weight=2)
G.add_edge('C', 'E', weight=1)
G.add_edge('D', 'E', weight=3)

# 计算从节点'A'到节点'E'的最短路径
shortest_path = nx.dijkstra_path(G, 'A', 'E', weight='weight')
print("最短路径:", shortest_path)

# 计算从节点'A'到节点'E'的最短路径长度
shortest_path_length = nx.dijkstra_path_length(G, 'A', 'E', weight='weight')
print("最短路径长度:", shortest_path_length)
```

2. Bellman-Ford算法

Bellman-Ford算法也是一种用于解决单源最短路径问题的算法，但它能够处理带有负权边的图。不过，它的时间复杂度较高，为 $O(VE)$ ，其中 V 是顶点数， E 是边数。在 `networkx` 中，可以使用 `bellman_ford_path` 和 `bellman_ford_path_length` 函数来计算最短路径和最短路径长度。

注意：由于Bellman-Ford算法的特性，它通常用于需要处理负权边的场景。

3. Floyd-Warshall算法

Floyd-Warshall算法是一种用于解决所有顶点对之间最短路径问题的动态规划算法。它能够处理带有正权、负权（但不能有负权回路）的图。在 `networkx` 中，可以使用 `floyd_warshall` 函数来计算图中所有顶点对之间的最短路径长度，但它不直接返回路径本身。如果需要路径，可以通过额外的步骤从最短路径长度矩阵中重建路径。

示例代码（仅计算最短路径长度）：

```
import networkx as nx

# 创建一个有向图
G = nx.DiGraph()
# 添加带权重的边
G.add_edge('A', 'B', weight=3)
# ... (省略其他边的添加)

# 计算所有顶点对之间的最短路径长度
path_lengths = nx.floyd_warshall_numpy(G, weight='weight')
print("最短路径长度矩阵:\n", path_lengths)

# 注意: Floyd-Warshall算法不直接返回路径, 但可以从path_lengths中重建路径
```

- 对于单源最短路径问题, 如果不存在负权边, 推荐使用Dijkstra算法 (`dijkstra_path` 和 `dijkstra_path_length`) 。
- 如果需要处理负权边, 则应使用Bellman-Ford算法 (`bellman_ford_path` 和 `bellman_ford_path_length`) 。
- 对于需要求解所有顶点对之间最短路径长度的问题, 可以使用Floyd-Warshall算法 (`floyd_warshall_numpy`) , 但请注意它不直接返回路径。

在实际应用中, 应根据问题的具体要求和图的特性选择合适的算法。